



Proyecto #2 – Nutrición para todos

Técnicas De Programación

Curso: 05.2026.IC.90501.G1.TECNICAS DE

PROGRAMACION

Fecha de entrega: 21 de Abril antes de las 6:00pm.

Estudiante: Cristian Casares

Contenidos

Introducción	5
Decisiones de diseño	5
Arquitectura del sistema.....	5
Uso de base de datos	7
Patrones y principios aplicados	8
Manejo de errores.....	9
Estrategia de testing	9
Desarrollo del sistema.....	10
Gestión de usuarios	10
Gestión de productos	11
Gestión de menús	11
Estadísticas	12
Exportación de reportes.....	14

Base de datos.....	15
Tablas principales.....	15
Relaciones	15
Ejecución del sistema.....	18
Requisitos	18
Pasos para ejecutar	18
Pruebas y resultados.....	19
Estrategia.....	19
Cobertura.....	19
Observaciones	20
Análisis de resultados, aprendizajes y conclusiones	21
Resultados	21
Aprendizajes.....	21
Conclusiones	22

Introducción

El presente documento describe el desarrollo de la segunda versión del sistema **Dragon Nutrex**, una aplicación orientada a la gestión nutricional de usuarios. Esta versión representa una mejora significativa con respecto al proyecto anterior, incorporando una arquitectura más robusta, persistencia en base de datos relacional, una interfaz web desarrollada con Blazor y nuevas funcionalidades orientadas a administración y análisis de datos.

El sistema permite gestionar usuarios, productos alimenticios, menús diarios y consumo nutricional, además de calcular requerimientos nutricionales y generar estadísticas relevantes. También se implementó la exportación de reportes en formatos CSV y TXT.

El desarrollo se realizó utilizando el lenguaje C#, siguiendo principios de programación orientada a objetos, buenas prácticas de Clean Code y una arquitectura basada en separación de responsabilidades.

Decisiones de diseño

Arquitectura del sistema

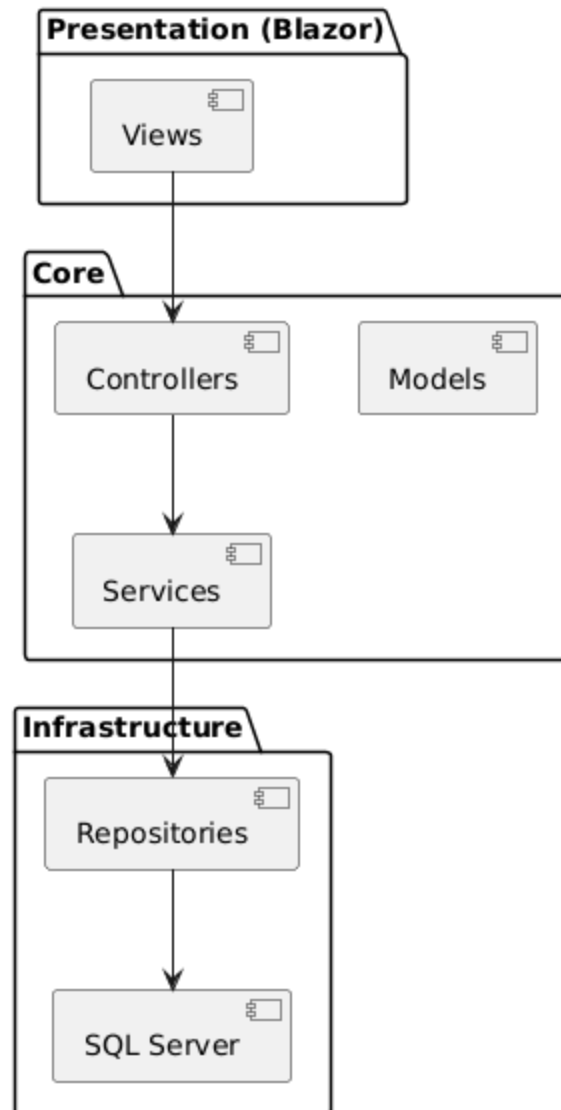
Se optó por una arquitectura en capas, con el objetivo de mejorar la mantenibilidad, escalabilidad y claridad del código. Las capas utilizadas fueron:

- Presentation (Blazor): encargada de la interfaz de usuario.

- Core: contiene la lógica de negocio, modelos, servicios y componentes de coordinación como el ConsumoController.
- Infrastructure: maneja el acceso a datos mediante repositorios y conexión a la base de datos.

Esta separación permite modificar o extender cada capa sin afectar directamente a las demás.

Arquitectura Dragon Nutrex Web



Uso de base de datos

A diferencia del proyecto anterior, donde los datos se manejaban mediante archivos, en esta versión se implementó una base de datos relacional en SQL Server.

Se utilizó el patrón Repository para abstraer el acceso a datos, lo que permite desacoplar la lógica de negocio de la persistencia. Cada entidad principal cuenta con su propio repositorio.

Patrones y principios aplicados

Durante el desarrollo se aplicaron principios de programación orientada a objetos y buenas prácticas de organización del código. La estructura del sistema se diseñó para separar responsabilidades y evitar que la interfaz, la lógica de negocio y el acceso a datos quedaran mezclados.

Uno de los patrones principales utilizados fue el Repository Pattern, aplicado en la capa de infraestructura para centralizar el acceso a la base de datos. De esta forma, los servicios de negocio no ejecutan directamente consultas SQL, sino que se comunican con repositorios encargados de obtener, crear, actualizar o eliminar información.

También se utilizó inyección de dependencias, configurada en Program.cs, para registrar servicios y repositorios. Esto permitió reducir el acoplamiento entre clases y facilitó la creación de pruebas unitarias, ya que las dependencias pudieron ser simuladas mediante mocks.

En la capa de presentación se utilizó Blazor, por lo que el sistema no sigue un MVC clásico con controladores y vistas tradicionales. En su lugar, se aplicó una arquitectura basada en componentes, donde cada vista o componente Razor se encarga de representar una parte específica de la interfaz y delega la lógica de negocio a los servicios correspondientes.

Manejo de errores

El manejo de errores se trabajó principalmente desde la capa de servicios, donde se incorporaron validaciones antes de ejecutar operaciones críticas. En lugar de permitir que datos inválidos llegaran hasta la base de datos, los servicios verifican condiciones como campos obligatorios, valores numéricos mayores a cero, fechas válidas y existencia previa de entidades.

Para mejorar la claridad del código y cumplir con buenas prácticas, se evitaron excepciones genéricas en la lógica principal. En su lugar, se utilizaron excepciones más específicas, como `ArgumentException`, `ArgumentOutOfRangeException` y `KeyNotFoundException`, según el tipo de error presentado.

Además, se incluyó soporte para registro de errores mediante `Logger`. La gestión principal de errores se realiza directamente en los servicios a través de validaciones y excepciones específicas.

Estrategia de testing

La estrategia de pruebas se enfocó en validar la lógica de negocio del sistema y no únicamente en verificar que los métodos se ejecutaran. Por esta razón, las pruebas se concentraron principalmente en los servicios del núcleo del sistema, donde se encuentran las reglas más importantes.

Se probaron escenarios positivos y negativos. Entre los casos positivos se validó la creación de usuarios, productos, menús, detalles de menú y cálculos nutricionales. En los casos negativos se

verificó que el sistema respondiera correctamente ante datos inválidos, como nombres vacíos, porciones menores o iguales a cero, fechas incorrectas o entidades inexistentes.

También se incluyeron pruebas para el ConsumoController, ya que este actúa como punto de coordinación entre los servicios de usuarios, nutrición y consumo. Para simular dependencias se utilizó Moq, principalmente sobre repositorios. Una limitación encontrada fue que Moq no permite simular directamente métodos no virtuales de clases concretas, por lo que en algunos casos se trabajó con servicios reales y repositorios simulados.

Desarrollo del sistema

Gestión de usuarios

El sistema permite administrar usuarios desde la vista correspondiente. El administrador puede crear nuevos usuarios, editar su información personal y nutricional, activar o desactivar cuentas y restablecer contraseñas.

La desactivación de usuarios se manejó como una alternativa más segura que la eliminación física directa, ya que permite conservar la información relacionada con menús, consumos y estadísticas. Esto evita conflictos con registros asociados y mantiene la integridad de los datos dentro de la base de datos.

Cada usuario cuenta con información nutricional relevante, como peso, altura, edad, nivel de actividad, objetivo nutricional y tipo de dieta. Estos datos son utilizados por el sistema para calcular requerimientos nutricionales personalizados.

Gestión de productos

La gestión de productos permite al administrador registrar y mantener los alimentos disponibles dentro del sistema. Para cada producto se almacena su nombre, categoría, porción en gramos, proteínas, carbohidratos, grasas y calorías.

El cálculo de calorías se realiza a partir de los macronutrientes, usando la equivalencia energética correspondiente: proteínas y carbohidratos aportan 4 kcal por gramo, mientras que las grasas aportan 9 kcal por gramo. Esto evita que el valor calórico dependa únicamente de una entrada manual.

También se implementó la activación y desactivación de productos. Esta decisión se tomó porque algunos productos pueden estar asociados a menús existentes, por lo que eliminarlos directamente podría generar conflictos de integridad referencial en la base de datos.

Gestión de menús

La gestión de menús permite crear menús diarios asociados a un usuario y a una fecha específica. Cada menú puede contener varios productos, y cada producto se registra dentro del menú mediante un detalle que indica la porción consumida.

La estructura utilizada permite separar el menú general de sus productos específicos. Para esto se utiliza la entidad MenuDetalle, que funciona como relación entre MenuDiario y Producto. Esta decisión facilita que un mismo producto pueda utilizarse en distintos menús sin duplicar información.

Además, se implementó la posibilidad de consultar menús existentes, agregar productos y eliminar detalles. En el caso de menús ya guardados, las modificaciones sobre los detalles se persisten para que los cambios no sean únicamente temporales.

Estadísticas

El sistema incorpora estadísticas administrativas orientadas al análisis general de la información registrada. Estas estadísticas permiten al administrador tener una visión más amplia del uso del sistema y de los hábitos registrados por los usuarios.

Se implementó la consulta del producto más consumido dentro de un rango de fechas. Esta funcionalidad utiliza los menús registrados y sus detalles para identificar cuál producto acumula la mayor cantidad de porciones.

También se calcula el porcentaje de tipos de dieta entre los usuarios registrados. Esta estadística permite observar la distribución de dietas dentro del sistema, por ejemplo, balanceada, cetogénica, alta en carbohidratos o baja en carbohidratos.

Finalmente, se incorporó una estadística de usuarios con más menús ingresados, que permite identificar cuáles usuarios han registrado mayor actividad dentro de la aplicación.

Dragon Nutrex

Gestión nutricional inteligente

Inicio

Usuarios

Productos

Menús

Estadísticas

IMC

Requerimientos

Stats Admin

Dragon Nutrex Web

Panel principal del sistema

Administrador (Admin)

Cerrar sesión

Estadísticas Administrativas

Consulta indicadores globales del sistema para administración.

Producto más consumido

Fecha inicio

15 / 04 / 2026

Fecha fin

22 / 04 / 2026

Consultar

Exportar CSV

Exportar TXT

No hay resultado disponible.

Porcentaje de tipos de dieta

Consultar

Exportar CSV

Exportar TXT

Tipo de dieta	Cantidad usuarios	Porcentaje
Balanceada	2	50 %
AltaEnCarbohidratos	1	25 %

Dragon Nutrex

Gestión nutricional inteligente

Inicio

Usuarios

Productos

Menús

Estadísticas

IMC

Requerimientos

Stats Admin

Producto más consumido

Consultar

Exportar CSV

Exportar TXT

No hay resultado disponible.

Porcentaje de tipos de dieta

Consultar

Exportar CSV

Exportar TXT

Tipo de dieta	Cantidad usuarios	Porcentaje
Balanceada	2	50 %
AltaEnCarbohidratos	1	25 %
BajaEnCarbohidratos	1	25 %

Usuarios con más menús ingresados

Consultar

Exportar CSV

Exportar TXT

Usuario	Cantidad de menús
Administrador	1
Carlos Ramirez	1
Maria López	1

Exportación de reportes

El sistema permite exportar reportes en formatos CSV y TXT. Esta funcionalidad se agregó para que la información consultada no quedara limitada únicamente a la pantalla, sino que pudiera descargarse y utilizarse externamente.

Los reportes incluyen información como consumo frente a meta, estadísticas por rango de fechas, producto más consumido, porcentaje de tipos de dieta y usuarios con más menús registrados.

El formato CSV facilita abrir los datos en herramientas como Excel, mientras que el formato TXT ofrece una versión más legible y directa para revisión rápida.

The screenshot displays the 'Dragon Nutrex Web' interface. On the left is a dark sidebar with the logo 'Dragon Nutrex' and the tagline 'Gestión nutricional inteligente'. The sidebar menu includes 'Inicio', 'Menús', 'Estadísticas' (highlighted in blue), 'IMC', and 'Requerimientos'. The main content area has a header with 'Dragon Nutrex Web', 'Panel principal del sistema', and a user profile for 'Carlos Ramirez (Usuario)' with a 'Cerrar sesión' button.

The main section is titled 'Estadísticas Nutricionales' with the subtitle 'Consulta el consumo diario frente a la meta y el resumen por rango de fechas.' A green success message states: 'Consulta de estadísticas por rango realizada correctamente.'

There are two main data entry sections:

- Consumo vs Meta:** Includes a 'Fecha' field with the value '22 / 04 / 2026' and a 'Consultar' button. Below it, a message says 'No hay registros para la consulta seleccionada.' Below this section are buttons for 'Exportar CSV' and 'Exportar TXT'.
- Estadísticas por Rango:** Includes 'Fecha inicio' (15 / 04 / 2026) and 'Fecha fin' (22 / 04 / 2026) fields, both with calendar icons, and a 'Consultar' button. Below this is a summary table of statistics:

Total calorías	Promedio diario	Total proteínas	Total carbohidratos	Total grasas	Días con registro
6250.00	2083.33	360.00	740.00	210.00	3

At the bottom of the 'Estadísticas por Rango' section are buttons for 'Exportar CSV' and 'Exportar TXT'.

Base de datos

Tablas principales

La segunda versión del sistema utiliza una base de datos relacional en SQL Server. Esta base de datos sustituyó el manejo de archivos utilizado en la primera versión del proyecto.

Las tablas principales son:

- Usuarios: almacena la información personal, credenciales básicas, rol, datos físicos y preferencias nutricionales.
- Productos: almacena los alimentos disponibles, su categoría, macronutrientes, porción, calorías y estado activo.
- MenusDiarios: almacena los menús creados por los usuarios en fechas específicas.
- MenuDetalles: almacena los productos asociados a cada menú y la porción correspondiente.
- ConsumosDiarios: almacena registros diarios de consumo nutricional para comparar contra las metas.

Relaciones

El diseño de la base de datos permite mantener relaciones claras entre las entidades principales del sistema. Estas relaciones se implementan mediante llaves foráneas en la base de datos, lo que asegura la integridad referencial entre las tablas.

- Un usuario puede tener varios menús diarios, por lo que existe una relación de uno a muchos entre Usuarios y MenusDiarios.
- Un menú diario puede tener varios detalles, por lo que existe una relación de uno a muchos entre MenusDiarios y MenuDetalles.
- Un producto puede aparecer en varios detalles de menú, por lo que existe una relación de uno a muchos entre Productos y MenuDetalles.
- La tabla MenuDetalles permite resolver la relación entre menús y productos sin duplicar la información nutricional de cada producto.

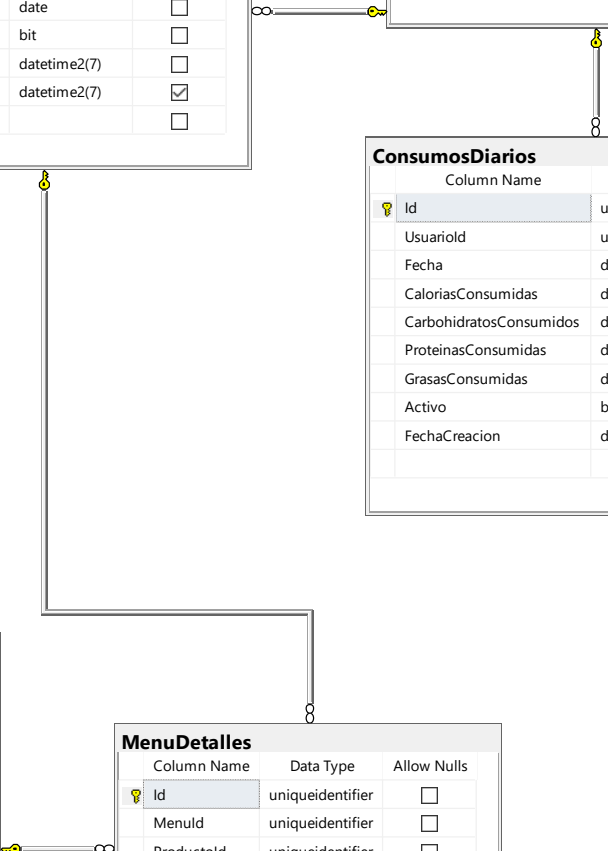
Usuarios		
Column Name	Data Type	Allow Nulls
 Id	uniqueidentifier	☐
Nombre	nvarchar(150)	☐
Correo	nvarchar(150)	☐
Contrasena	nvarchar(200)	☐
Rol	nvarchar(20)	☐
Peso	decimal(5, 2)	☐
Altura	decimal(4, 2)	☐
Edad	int	☐
NivelActividad	nvarchar(30)	☐
Objetivo	nvarchar(30)	☐
TipoDieta	nvarchar(30)	☐
Activo	bit	☐
FechaCreacion	datetime2(7)	☐
FechaActualizacion	datetime2(7)	☒
		☐

MenusDiarios		
Column Name	Data Type	Allow Nulls
 Id	uniqueidentifier	☐
Usuariold	uniqueidentifier	☐
Nombre	nvarchar(150)	☐
Fecha	date	☐
Activo	bit	☐
FechaCreacion	datetime2(7)	☐
FechaActualizacion	datetime2(7)	☒
		☐

ConsumosDiarios		
Column Name	Data Type	Allow Nulls
 Id	uniqueidentifier	☐
Usuariold	uniqueidentifier	☐
Fecha	date	☐
CaloriasConsumidas	decimal(10, 2)	☐
CarbohidratosConsumidos	decimal(10, 2)	☐
ProteinasConsumidas	decimal(10, 2)	☐
GrasasConsumidas	decimal(10, 2)	☐
Activo	bit	☐
FechaCreacion	datetime2(7)	☐
		☐

Productos		
Column Name	Data Type	Allow Nulls
 Id	uniqueidentifier	☐
Nombre	nvarchar(150)	☐
Categoria	nvarchar(50)	☐
Proteina	decimal(8, 2)	☐
Carbohidratos	decimal(8, 2)	☐
Grasas	decimal(8, 2)	☐
PorcionGramos	decimal(8, 2)	☐
Calorias	decimal(10, 2)	☐
Activo	bit	☐
FechaCreacion	datetime2(7)	☐
FechaActualizacion	datetime2(7)	☒
		☐

MenuDetalles		
Column Name	Data Type	Allow Nulls
 Id	uniqueidentifier	☐
MenuId	uniqueidentifier	☐
Productold	uniqueidentifier	☐
Porcion	decimal(8, 2)	☐
FechaCreacion	datetime2(7)	☐
		☐



Ejecución del sistema

Requisitos

Para ejecutar el sistema se requiere:

- .NET 10.
- SQL Server.
- Visual Studio.
- Cadena de conexión configurada en appsettings.json.

Pasos para ejecutar

1. Clonar el repositorio desde Git.
2. Abrir la solución en Visual Studio.
3. Verificar que la base de datos exista en SQL Server.
4. Configurar la cadena de conexión en el archivo appsettings.json.
5. Establecer el proyecto web como proyecto de inicio.
6. Ejecutar la aplicación desde Visual Studio o mediante el comando: `dotnet run`
7. Acceder a la aplicación desde el navegador en la dirección indicada por la consola o por Visual Studio.

Pruebas y resultados

Estrategia

Las pruebas unitarias se enfocaron en la lógica de negocio y en el controlador de consumo (ConsumoController). No se probaron directamente las vistas Razor ni los repositorios SQL, ya que el objetivo principal fue validar las reglas del sistema sin depender de la interfaz ni de una base de datos real.

Se probaron servicios como:

- UsuarioService
- ProductoService
- MenuDiarioService
- MenuDetalleService
- ConsumoService
- NutricionService
- AdminEstadisticasService

También se incluyeron pruebas para ConsumoController.

Cobertura

La cobertura se midió con Coverlet, aplicando una configuración enfocada en la capa de negocio. Se excluyeron capas como presentación e infraestructura, ya que estas no formaban parte del alcance principal de las pruebas unitarias.

Los resultados obtenidos fueron:

- Cobertura total: 73.1%
- Cobertura de métodos: 80.8%
- Cobertura de ramas: 63.8%

Estos resultados superan el mínimo de 60% solicitado para la evaluación del proyecto.

Herramientas utilizadas

- xUnit
- Moq
- Coverlet
- SonarAnalyzer.CSharp

Observaciones

Durante las pruebas se validaron tanto escenarios correctos como escenarios de error. Por ejemplo, se comprobó que el sistema rechazara porciones inválidas, usuarios inexistentes, productos inexistentes y fechas incorrectas.

Una limitación encontrada fue que Moq no permite simular métodos no virtuales de clases concretas. Por eso, en algunos casos se utilizaron servicios reales conectados a repositorios simulados.

Análisis de resultados, aprendizajes y conclusiones

Resultados

Como resultado de esta segunda versión, se obtuvo una aplicación web funcional desarrollada con Blazor. El sistema permite administrar usuarios, productos, menús, estadísticas y reportes descargables.

También se logró migrar el almacenamiento de archivos a una base de datos relacional, lo cual representa una mejora importante en organización, seguridad e integridad de los datos.

Además, se incorporaron pruebas unitarias y análisis de calidad con SonarAnalyzer, lo que permitió identificar y corregir malas prácticas como excepciones genéricas y nombres de parámetros inconsistentes.

Aprendizajes

Durante el desarrollo se fortaleció el uso de arquitectura en capas, inyección de dependencias y separación de responsabilidades. También se reforzó la importancia de diseñar servicios que sean fáciles de probar y mantener.

El uso de Blazor permitió trabajar una interfaz web sin abandonar el ecosistema de C#. Además, el proceso de pruebas mostró la importancia de escribir código desacoplado para facilitar la simulación de dependencias.

Otro aprendizaje importante fue que las herramientas de análisis como Sonar no deben seguirse de forma automática. Algunas sugerencias son útiles, pero otras deben evaluarse según el diseño general del sistema.

Conclusiones

La versión 2 del sistema Dragon Nutrex representa una mejora significativa respecto a la primera versión. La aplicación pasó de una solución basada en archivos a una aplicación web con base de datos, arquitectura en capas, administración de usuarios, estadísticas y reportes descargables.

El sistema quedó mejor organizado y preparado para futuras extensiones. Aunque todavía existen posibles mejoras, como ampliar pruebas sobre autenticación o exportación de reportes, la funcionalidad principal cumple con los requerimientos establecidos para esta iteración.